

Community detection (CD) - regular assignment

Most useful link: Chapter 5 tutorial from "Mining Complex Networks":

https://github.com/ftheberge/GraphMiningNotebooks/blob/master/Python_Notebooks_Second_Edition/Chapter_5.ipynb

Task 1 (2p.): Compare at least 3 different community detection algorithms for 2 different datasets (graphs). Compute the partitions and compare their modularity.

Modularity function implementation:

<https://python.igraph.org/en/stable/api/igraph.GraphBase.html#modularity>

Examples of datasets and algorithms are listed at the end of this document

Task 2a* (1.5p.): Compare at least 3 different community detection algorithms for 2 different datasets given as a graph (different from the ones used in Task 1) for which the ground-truth partition is known. Compute the partitions and compare their modularity as well as AMI and ARI measures.

AMI: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.adjusted_mutual_info_score.html

ARI: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.adjusted_rand_score.html

Examples of datasets and algorithms are listed at the end of this document

Task 2b* (1.5p). Generate at least 3 synthetic networks with the use of the ABCD benchmark

<https://github.com/bkamins/ABCDGraphGenerator.jl>, and compare at least 3 CD algorithms with the use of modularity and AMI measure

Outcome: the outcome files with edges and ground-truth partition, the Julia commands used to generate hypergraphs, and the notebook). Check if the CD algorithms give deterministic results or have some random elements.

Task 2c* (1p). Provide the Bayesian optimization

(e.g. based on <https://github.com/bayesian-optimization/BayesianOptimization>)

for the selected hyperparameter of one of the algorithms. A good example is the *resolution* parameter of an igraph implementation of the Leiden algorithm

(https://python.igraph.org/en/stable/api/igraph.GraphBase.html#community_leiden).

Task 2d* (1p.): Compare at least 3 different community detection algorithms for 2 different hypergraph datasets. Transform hypergraphs to their 2-section graph, use the graph-based CD algorithm to compute the partitions, and compare their hypergraph modularity (please provide the comparison for strict, linear and majority modularity).

modularity functions' implementation:

https://github.com/pnnl/HyperNetX/blob/master/hypernetx/algorithms/hypergraph_modularity.py

* additionally you may use hypergraph-based CD algorithms from hypernetx (Kumar and last_step):

https://hypernetx.readthedocs.io/en/latest/algorithms/hypergraph_modularity_and_clustering.html

Task 2f* (1p). Using the ABCDh benchmark

<https://github.com/bkamins/ABCDHypergraphGenerator.jl/tree/main>,

generate at least 2 synthetic hypergraph networks (on 300 nodes) and compare at least 3 CD algorithms using modularity and AMI measures.

Outcome: the outcome files with edges (`_he`) and ground truth (`_assign`), the Julia commands used to generate hypergraphs and the notebook.

Useful links:

CD Algorithms that you may use:

GRAPHS:

- Louvain
https://python.igraph.org/en/stable/api/igraph.GraphBase.html#community_multilevel
- Leiden
https://python.igraph.org/en/stable/api/igraph.GraphBase.html#community_leiden
- Grivan-Newman (community_edge_betweenness)
https://python.igraph.org/en/stable/api/igraph.GraphBase.html#community_edge_betweenness
- ECG
<https://pypi.org/project/partition-igraph/>
- Label Propagation
https://python.igraph.org/en/stable/api/igraph.GraphBase.html#community_label_propagation
- CNM
https://python.igraph.org/en/stable/api/igraph.GraphBase.html#community_fastgreedy
- Infomap
https://python.igraph.org/en/stable/api/igraph.GraphBase.html#community_infomap

HYPERGRAPHS:

- Kumar (improvement of Louvain on the two-section graph)
https://github.com/pnnl/HyperNetX/blob/master/hypernetx/algorithms/hypergraph_modularity.py
- Last_step - hypergraph-based improvement of the already proposed partition
https://github.com/pnnl/HyperNetX/blob/master/hypernetx/algorithms/hypergraph_modularity.py

Datasets, that you may use:

GRAPHS:

- Stanford repository
<https://snap.stanford.edu/data/>
- Datasets linked at *Dataset Reference* section of:
<https://github.com/ftheberge/GraphMiningNotebooks/tree/master>
- Austin Benson's repository:
<https://www.cs.cornell.edu/~arb/data/>

HYPERGRAPHS:

- Hypernetx basic datasets (GoT, LessMiss, etc.):
<https://github.com/pnnl/HyperNetX/tree/master/hypernetx/utis/toys>
- Austin Benson's repository:
<https://www.cs.cornell.edu/~arb/data/>
- XGI repository:
<https://xgi.readthedocs.io/en/stable/xgi-data.html>